Solve Any Data Analysis Problem

Chapter 3 - Project 2: Who are your customers?

Goals

- **explore** all 3 datasets
- **come up with a common schema** to record customers (i.e. what columns of customer data are common across data sources?)
- **populate** new customer dataset with:
  - customers who have made a purchase and can be identified in the purchases table, and looking up their details in the customer datasets
  - actual customers from the customer database (if they aren't already referenced in the purchases table)
  - customers from CRM data (if they aren't already referenced in the purchases table)
  - guest checkouts from purchase history
- for each "inferred" customer record, **store**:
  - all relevant customer data
  - source of information
- **deduplicate** dataset

Final output

A dataset containing one row per "inferred customer entity".

Part 1 - exploring the data

In [1]:
```
import pandas as pd
import numpy as np
```

Sales

In [2]:
```
sales = pd.read_csv("./data/purchases.csv")
print(sales.shape)
```

```
(71519, 11)
```

In [3]:
```
sales.head()
```

Out[3]:

	event_time	product_id	category_id	category_code	brand	pric
0	2022-10-01 02:26:08+00:00	32701106	2055156924466332447	NaN	shimano	95.2
1	2022-10-01 02:28:32+00:00	9400066	2053013566067311601	NaN	jaguar	164.2
2	2022-10-01 02:31:01+00:00	1004238	2053013555631882655	electronics.smartphone	apple	1206.4
3	2022-10-01 02:33:31+00:00	11300059	2053013555531219353	electronics.telephone	texet	17.4
4	2022-10-01 02:40:18+00:00	17300751	2053013553853497655	NaN	versace	77.2

Let's check for missing values

In [4]:
```
sales.isnull().sum()
```

Out[4]:
```
event_time             0
product_id             0
category_id            0
category_code      16739
brand               5707
price                  0
session_id             0
customer_id        18448
guest_first_name   53071
guest_surname      53071
guest_postcode     53071
dtype: int64
```

Looks like the 18448 missing customer IDs (which are guest checkouts) and the 53071 missing guest values (which are registered customers) make up all of our data. We should verify there's no overlap, e.g. a row with both customer ID and guest details missing.

Let's create a column to track guest checkouts:

In [5]:
```
sales["is_guest"] = sales["customer_id"].isnull()
```

Are there cases where guest checkouts also had a customer ID filled in?

In [6]:
```
sales[sales["is_guest"] & sales["customer_id"].notnull()]
```

Out[6]:

	event_time	product_id	category_id	category_code	brand	price	session_id	customer_i

What about anywhere with **neither**?

In [7]:
```
sales[(sales["is_guest"] == False) & sales["customer_id"].isnull()]
```

Out[7]:

	event_time	product_id	category_id	category_code	brand	price	session_id	customer_i

Let's look at the proportion of guest vs. non-guest checkouts:

In [8]:
```
sales["is_guest"].value_counts(normalize=True)
```

Out[8]:
```
False    0.742055
True     0.257945
Name: is_guest, dtype: float64
```

Approx. 26% are guest checkouts. How many unique customer names does that translate to?

In [9]:
```
print("{} rows of purchases with guest checkout".format(len(sales[sales["is_guest"]
```

```
18448 rows of purchases with guest checkout
```

In [10]:
```
guest_columns = ["guest_first_name", "guest_surname", "guest_postcode"]
unique_guests = sales[guest_columns].drop_duplicates()
print(len(unique_guests))
unique_customers = sales["customer_id"].unique()
cust_total = len(unique_customers) + len(unique_guests)
print(len(unique_guests) / (cust_total-1))
```

```
8301
0.2495640671036017
```

In [11]:
```
sales["customer_id"].nunique()
```

Out[11]:
```
24961
```

Nearly 25k customers make up the other 75%.

Sanity check: do we have any guest checkouts with a customer ID or a row with neither?

In [12]:
```
missing_guest_info = (
    sales[(sales["customer_id"].isnull()) &
              ((sales["guest_first_name"].isnull())
               | (sales["guest_surname"].isnull())
               | (sales["guest_postcode"].isnull()))]
)

print(len(missing_guest_info))
missing_guest_info
```

```
0
```

Out[12]:

event_time	product_id	category_id	category_code	brand	price	session_id	customer_i

No.

And no records where we have a customer ID, but also guest information

Summary: purchases

- we have ~25k unique customers
- plus ~18k records with guest checkouts, which is roughly 8k unique guest customers (not counting duplicates)

Total expected customer base (from transactions): **approx. 30-35k**

Customer data avaliable (from guest checkouts):

- first name
- surname
- postcode

Let's export this to its own DataFrame to merge with the other customers later:

In [13]:
```
guest_columns = ["guest_first_name", "guest_surname", "guest_postcode", "is_guest"]
guests = sales.loc[sales["is_guest"], guest_columns].drop_duplicates()
guests.head()
```

Out[13]:

	guest_first_name	guest_surname	guest_postcode	is_guest
0	MICHAEL	MASON	RG497ZQ	True
2	COLE	WILKINSON	SW75TQ	True
3	MOHAMMED	RICHARDS	RG150RE	True
7	KIAN	MILLS	SW332TF	True
13	RUBY	OWEN	PO377YS	True

In [14]:
```
non_guests = (
    pd.DataFrame(
        sales.loc[sales["customer_id"].notnull(), "customer_id"]
            .unique()
            .astype(int),
        columns=["customer_id"])
)

non_guests.head()
```

Out[14]:

	customer_id
0	7466
1	31266
2	534142828
3	1035
4	6985

Merge the two (accepting that we'll have NULLs)

In [15]:
```
sales_customers = pd.concat([non_guests, guests], axis=0, ignore_index=True)
new_col_names = ["customer_id", "first_name", "surname", "postcode", "is_guest"]
sales_customers = sales_customers.set_axis(new_col_names, axis=1) # rename columns

sales_customers["is_guest"] = sales_customers["is_guest"].fillna(False)

# mark the source of the records
sales_customers["in_purchase_data"] = True
```

Let's sanitize the name columns:

In [16]:
```
for col in ["first_name", "surname"]:
    sales_customers[col] = sales_customers[col].str.lower().str.strip()

sales_customers["postcode"] = sales_customers["postcode"].str.strip()

sales_customers
```

Out[16]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data
0	7466.0	NaN	NaN	NaN	False	True
1	31266.0	NaN	NaN	NaN	False	True
2	534142828.0	NaN	NaN	NaN	False	True
3	1035.0	NaN	NaN	NaN	False	True
4	6985.0	NaN	NaN	NaN	False	True
...	...	...	...	...	...	...
33256	NaN	poppy	foster	M192EQ	True	True
33257	NaN	sophie	chapman	NW500AS	True	True
33258	NaN	scarlett	shaw	EX86QS	True	True
33259	NaN	michael	harrison	HR280TG	True	True
33260	NaN	skye	green	HR478ER	True	True

33261 rows × 6 columns

CRM data

In [17]:
```
crm = pd.read_csv("./data/crm_export.csv")
print(crm.shape)
crm.head()
```

```
(7825, 5)
```

Out[17]:

	customer_id	first_name	surname	postcode	age
0	29223	Holly	Rogers	LS475RT	12
1	27826	Daniel	Owen	M902XX	5
2	7432	Eleanor	Russell	HR904ZA	34
3	2569	Paige	Roberts	DE732EP	61
4	9195	Matilda	Young	LS670FU	78

In [18]:
```
crm.isnull().sum()
```

Out[18]:
```
customer_id    0
first_name     0
surname        0
postcode       0
age            0
dtype: int64
```

Let's sanitize the data so strings are all lowercase and whitespace is removed

In [19]:
```
for col in ["first_name", "surname"]:
    crm[col] = crm[col].str.lower().str.strip()

crm["postcode"] = crm["postcode"].str.strip()
```

Let's verify that customer ID is in fact unique

In [20]:
```
crm.groupby("customer_id").size().loc[lambda x: x > 1]
```

Out[20]:
```
Series([], dtype: int64)
```

No. What about duplicate information?

In [21]:
```
print(len(crm))
print(len(crm.drop(columns="customer_id").drop_duplicates()))
```

```
7825
7419
```

In [22]:
```
print("{} rows".format(len(crm)))
print("{} unique combinations of customer information".format(len(crm.drop(columns=
```

```
7825 rows
7419 unique combinations of customer information
```

Now, join CRM data to purchases where purchases have a customer ID (not guests)

In [23]:
```
print(len(sales_customers))
sales_customers.isnull().sum()
```

```
33261
```

Out[23]:
```
customer_id          8300
first_name          24961
surname             24961
postcode            24961
is_guest                0
in_purchase_data        0
dtype: int64
```

In [24]:
```
sales_and_crm_customers = sales_customers.merge(crm, on="customer_id", how="left")
print(len(sales_and_crm_customers))
sales_and_crm_customers.isnull().sum()
```

```
33261
```

Out[24]:
```
customer_id          8300
first_name_x        24961
surname_x           24961
postcode_x          24961
is_guest                0
in_purchase_data        0
first_name_y        26147
surname_y           26147
postcode_y          26147
age                 26147
dtype: int64
```

Update record source for merged records

In [25]:
```
# now we have duplicate customer detail columns, so merge them into the old ones
merged_customers_filter = (
    (sales_and_crm_customers["customer_id"].notnull()) # only for actual customers
    # only if they have at least first or surname filled in
    # meaning we've found a match in our CRM data
    & ((sales_and_crm_customers["first_name_y"].notnull())
       | (sales_and_crm_customers["surname_y"].notnull()))
)

sales_and_crm_customers.loc[merged_customers_filter, "in_crm_data"] = True
sales_and_crm_customers.loc[~merged_customers_filter, "in_crm_data"] = False

sales_and_crm_customers["in_crm_data"].value_counts()
```

Out[25]:
```
False    26147
True      7114
Name: in_crm_data, dtype: int64
```

Now merge data into single versions of customer information

In [26]:
```
sales_and_crm_customers.loc[merged_customers_filter, ["first_name_x", "surname_x",
    sales_and_crm_customers.loc[merged_customers_filter, ["first_name_y", "surname_
```

```
    .values
)

# drop duplicate columns and rename
sales_and_crm_customers = (
    sales_and_crm_customers
    .drop(columns=["first_name_y", "surname_y", "postcode_y"])
    .rename(columns={
        "first_name_x": "first_name",
        "surname_x": "surname",
        "postcode_x": "postcode"
    })
)

print(sales_and_crm_customers.isnull().sum())
sales_and_crm_customers.head()
```

```
customer_id          8300
first_name          17847
surname             17847
postcode            17847
is_guest                0
in_purchase_data        0
age                 26147
in_crm_data             0
dtype: int64
```

Out[26]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_crm_c
0	7466.0	NaN	NaN	NaN	False	True	NaN	F
1	31266.0	harley	palmer	HR250EJ	False	True	33.0	
2	534142828.0	NaN	NaN	NaN	False	True	NaN	F
3	1035.0	NaN	NaN	NaN	False	True	NaN	F
4	6985.0	NaN	NaN	NaN	False	True	NaN	F

We now need to add CRM customers who are **not** in our purchase history.

In [27]:
```
crm_ids_to_add = list(set(crm["customer_id"].unique()) - set(sales_and_crm_customer
print(len(crm_ids_to_add))
```

```
711
```

In [28]:
```
print(len(sales_and_crm_customers))

sales_and_crm_customers = (
    pd.concat([sales_and_crm_customers, crm[crm["customer_id"].isin(crm_ids_to_add)
               axis=0,
              ignore_index=True)
)

print(len(sales_and_crm_customers))
```

```
33261
33972
```

In [29]:
```
sales_and_crm_customers.isnull().sum()
```

Out[29]:
```
customer_id          8300
first_name          17847
surname             17847
postcode            17847
is_guest              711
in_purchase_data      711
age                 26147
in_crm_data           711
dtype: int64
```

Update missing record sources accordingly

In [30]:
```
sales_and_crm_customers["is_guest"] = sales_and_crm_customers["is_guest"].fillna(Fa
sales_and_crm_customers["in_purchase_data"] = sales_and_crm_customers["in_purchase_
sales_and_crm_customers["in_crm_data"] = sales_and_crm_customers["in_crm_data"].fil

sales_and_crm_customers.isnull().sum()
```

Out[30]:
```
customer_id          8300
first_name          17847
surname             17847
postcode            17847
is_guest                0
in_purchase_data        0
age                 26147
in_crm_data             0
dtype: int64
```

In [31]:
```
sales_and_crm_customers["in_crm_data"].value_counts()
```

Out[31]:
```
False    26147
True      7825
Name: in_crm_data, dtype: int64
```

In [32]:
```
print(len(sales_and_crm_customers))
sales_and_crm_customers.head()
```

33972

Out[32]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_crm_c
0	7466.0	NaN	NaN	NaN	False	True	NaN	F
1	31266.0	harley	palmer	HR250EJ	False	True	33.0	
2	534142828.0	NaN	NaN	NaN	False	True	NaN	F
3	1035.0	NaN	NaN	NaN	False	True	NaN	F
4	6985.0	NaN	NaN	NaN	False	True	NaN	F

This is now one row per customer from BOTH the CRM data and purchase history including:

- guest checkouts from the purchases table

- customers in the purchases table who had a customer ID now have their data included (if they were in the CRM data)
- customers who are in the CRM database but have made no purchases

Customers

In [33]:
```
customers = pd.read_csv("./data/customer_database.csv")
print(customers.shape)
customers.head()
```

```
(23476, 5)
```

Out[33]:

	customer_id	first_name	surname	postcode	age
0	1641	Rhys	Richards	DE456EZ	45
1	24796	Maisie	Young	SW433XX	16
2	14358	Nathan	King	NW49TU	58
3	15306	Jack	Moore	NW908RR	26
4	24971	Alexander	Roberts	SW500HW	85

In [34]:
```
customers.isnull().sum()
```

Out[34]:
```
customer_id    0
first_name     0
surname        0
postcode       0
age            0
dtype: int64
```

Similar sanitizing to CRM:

In [35]:
```
for col in ["first_name", "surname"]:
    customers[col] = customers[col].str.lower().str.strip()

customers["postcode"] = customers["postcode"].str.strip()
```

In [36]:
```
customers.groupby("customer_id").size().loc[lambda x: x>1]
```

Out[36]:
```
Series([], dtype: int64)
```

In [37]:
```
print("{} rows".format(len(customers)))
unique_customers = customers.drop(columns="customer_id").drop_duplicates()
print("{} unique combinations of customers".format(len(unique_customers)))
```

```
23476 rows
19889 unique combinations of customers
```

Join this data to our growing merged customer data

In [38]:
```
print(len(sales_and_crm_customers))
```

```
all_customers = sales_and_crm_customers.merge(customers, on="customer_id", how="lef
print(len(all_customers))
all_customers.head()
```

```
33972
33972
```

Out[38]:

	customer_id	first_name_x	surname_x	postcode_x	is_guest	in_purchase_data	age_x	i
0	7466.0	NaN	NaN	NaN	False	True	NaN	
1	31266.0	harley	palmer	HR250EJ	False	True	33.0	
2	534142828.0	NaN	NaN	NaN	False	True	NaN	
3	1035.0	NaN	NaN	NaN	False	True	NaN	
4	6985.0	NaN	NaN	NaN	False	True	NaN	

In [39]:
```
# now we have duplicate customer detail columns, so merge them into the old ones
merged_customers_filter = (
    (all_customers["customer_id"].notnull()) # only for actual customers
    # only if they have at least first or surname filled in
    & ((all_customers["first_name_y"].notnull())
       | (all_customers["surname_y"].notnull()))
)

all_customers.loc[merged_customers_filter, "in_customer_data"] = True
all_customers.loc[~merged_customers_filter, "in_customer_data"] = False

all_customers["in_customer_data"].value_counts()
```

Out[39]:
```
True     22053
False    11919
Name: in_customer_data, dtype: int64
```

In [40]:
```
all_customers.isnull().sum()
```

Out[40]:
```
customer_id          8300
first_name_x        17847
surname_x           17847
postcode_x          17847
is_guest                0
in_purchase_data        0
age_x               26147
in_crm_data             0
first_name_y        11919
surname_y           11919
postcode_y          11919
age_y               11919
in_customer_data        0
dtype: int64
```

Fill in customer information if newly added

In [41]:
```
update_filter = (
```

```
    (all_customers["in_customer_data"])
    & (all_customers["first_name_x"].isnull())
    & (all_customers["surname_x"].isnull())
)

print(len(all_customers))

all_customers.loc[update_filter, ["first_name_x", "surname_x", "postcode_x", "age_x
    all_customers.loc[update_filter, ["first_name_y", "surname_y", "postcode_y", "a
)

all_customers = (
    all_customers
    .drop(columns=["first_name_y", "surname_y", "age_y", "postcode_y"])
    .rename(columns={
        "first_name_x": "first_name",
        "surname_x": "surname",
        "age_x": "age",
        "postcode_x": "postcode"
    })
)

print(len(all_customers))
all_customers.isnull().sum()
```

```
33972
33972
```

Out[41]:
```
customer_id         8300
first_name          1248
surname             1248
postcode            1248
is_guest               0
in_purchase_data       0
age                 9548
in_crm_data            0
in_customer_data       0
dtype: int64
```

Add customers from customer DB but not in main data

In [42]:
```
customer_ids_to_add = list(set(customers["customer_id"].unique()) - set(all_custome
print(len(customer_ids_to_add))
```

```
1423
```

In [43]:
```
print(len(all_customers))

all_customers = (
    pd.concat([all_customers, customers[customers["customer_id"].isin(customer_ids_
              axis=0,
              ignore_index=True)
)

print(len(all_customers))
```

```
33972
35395
```

In [44]:
```
all_customers.isnull().sum()
```

Out[44]:
```
customer_id         8300
first_name          1248
surname             1248
postcode            1248
is_guest            1423
in_purchase_data    1423
age                 9548
in_crm_data         1423
in_customer_data    1423
dtype: int64
```

Ensure source data is correct.

At this point if we have missing data for whether someone was a guest, it means they weren't (since we can only mark guests using the purchase data and we've already done that)

In [45]:
```
all_customers["is_guest"] = all_customers["is_guest"].fillna(False)
all_customers["in_purchase_data"] = all_customers["in_purchase_data"].fillna(False)
all_customers["in_crm_data"] = all_customers["in_crm_data"].fillna(False)
all_customers["in_customer_data"] = all_customers["in_customer_data"].fillna(True)

all_customers.isnull().sum()
```

Out[45]:
```
customer_id         8300
first_name          1248
surname             1248
postcode            1248
is_guest               0
in_purchase_data       0
age                 9548
in_crm_data            0
in_customer_data       0
dtype: int64
```

In [46]:
```
all_customers.head()
```

Out[46]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_crm_c
0	7466.0	eve	richards	HR90PT	False	True	45.0	F
1	31266.0	harley	palmer	HR250EJ	False	True	33.0	
2	534142828.0	NaN	NaN	NaN	False	True	NaN	F
3	1035.0	luca	gibson	DE256NH	False	True	30.0	F
4	6985.0	mia	rogers	HR662RP	False	True	43.0	F

In [47]:
```
len(all_customers)
```

Out[47]:
```
35395
```

Deduplicate

Easy one first, remove all exact duplicates (which might have happened because we exported data from purchases where one row is a purchase, not a customer):

In [48]:
```
print(len(all_customers))
all_customers = all_customers.drop_duplicates()
print(len(all_customers))
```

```
35395
35395
```

Now we have all possible customer records in one place including:

- those purchased with a customer ID (whether or not that ID is in another database)
- those purchased as guests
- those in the CRM database who haven't made a purchase
- those in the customer database who haven't made a purchase

Let's count up all these eventualities:

In [49]:
```
identified_customers = (
    all_customers[(all_customers["customer_id"].notnull())
                  & (all_customers["in_purchase_data"])
                  & ((all_customers["in_crm_data"])
                     | (all_customers["in_customer_data"]))]
)

guests = all_customers[all_customers["is_guest"]]

customer_ids_not_found = (
    all_customers[(all_customers["customer_id"].notnull())
                  & (all_customers["first_name"].isnull())
                  & (all_customers["surname"].isnull())]
)

customer_data_only = (
    all_customers[((all_customers["in_crm_data"])
                   | (all_customers["in_customer_data"])
                  )
                  & (all_customers["in_purchase_data"] == False)]
)

print(len(all_customers), len(identified_customers))
print(len(guests), len(customer_ids_not_found), len(customer_data_only))
```

```
35395 23713
8300 1248 2134
```

In [50]:
```
print(f"Size of customer database (not yet deduplicated): {len(all_customers)}")
print(f"Identified customers (with an ID, present in a lookup DB): {len(identified_
print(f"Guest checkouts: {len(guests)}")
print(f"Unidentified customer IDs in purchases: {len(customer_ids_not_found)}")
print(f"Customers with no purchases present in CRM or customer data: {len(customer_
```

```
Size of customer database (not yet deduplicated): 35395
Identified customers (with an ID, present in a lookup DB): 23713
Guest checkouts: 8300
Unidentified customer IDs in purchases: 1248
Customers with no purchases present in CRM or customer data: 2134
```

In [51]:
```
assert len(all_customers) == len(identified_customers) + len(guests) + len(customer
```

So we're roughly looking at 35,000 customers. We now need to deal with two types of duplication:

- different customer IDs encoding **exactly the same** information (exact duplicates)
- different customer IDs encoding **almost** exactly the same information (fuzzy duplicates)

When we identify our duplicates we can consolidate them into a "main" record which will be our best guess at a customer database.

How do we identify a duplicate?

- different customer ID (or NO customer ID in the case of guests)
- but the same details
  - first name
  - surname
  - postcode
  - age, **but** this isn't provided for guest accounts!

We want to have a "main" record where we also record all the IDs that we think refer to the same account.

Guests don't have IDs, let's give them some.

In [52]:
```
all_customers["customer_id"].agg(["min", "max"])
```

Out[52]:
```
min            1.0
max    566239774.0
Name: customer_id, dtype: float64
```

The range of existing IDs is quite large, maybe we're safer with negative numbers for guests (we could also go to a much higher range but some of the IDs are already 9 digits).

In [53]:
```
all_guests = all_customers[all_customers["is_guest"]].copy()

# go from -1 to -N as needed
new_ids = np.arange(-1, -(len(all_guests) + 1), -1)

all_customers.loc[all_customers["is_guest"], "customer_id"] = new_ids
all_customers.head()
```

Out[53]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_crm_c
0	7466.0	eve	richards	HR90PT	False	True	45.0	F
1	31266.0	harley	palmer	HR250EJ	False	True	33.0	
2	534142828.0	NaN	NaN	NaN	False	True	NaN	F
3	1035.0	luca	gibson	DE256NH	False	True	30.0	F
4	6985.0	mia	rogers	HR662RP	False	True	43.0	F

In [54]:
```
all_customers[all_customers["is_guest"]].head()
```

Out[54]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in
24961	-1.0	michael	mason	RG497ZQ	True	True	NaN	
24962	-2.0	cole	wilkinson	SW75TQ	True	True	NaN	
24963	-3.0	mohammed	richards	RG150RE	True	True	NaN	
24964	-4.0	kian	mills	SW332TF	True	True	NaN	
24965	-5.0	ruby	owen	PO377YS	True	True	NaN	

Let's convert customer ID to an integer to make it easier to use

In [55]:
```
all_customers["customer_id"] = all_customers["customer_id"].astype(int)
```

Now we try to deduplicate everything including guest accounts

In [56]:
```
columns_to_consider = ["first_name", "surname", "postcode"]

duplicates = all_customers[all_customers.duplicated(subset=columns_to_consider, kee

# Create a dictionary mapping duplicates to customer IDs
duplicate_dict = duplicates.groupby(columns_to_consider)['customer_id'].apply(list)

# Add a new column for the duplicated customer IDs
all_customers['other_customer_ids'] = all_customers.apply(lambda x: duplicate_dict.

all_customers
```

Out[56]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_
0	7466	eve	richards	HR90PT	False	True	45.0	
1	31266	harley	palmer	HR250EJ	False	True	33.0	
2	534142828	NaN	NaN	NaN	False	True	NaN	
3	1035	luca	gibson	DE256NH	False	True	30.0	
4	6985	mia	rogers	HR662RP	False	True	43.0	
...	...	...	...	...	...	...	...	
35390	14295	erin	morgan	NW481EN	False	False	63.0	
35391	28025	aaron	harris	SO265RP	False	False	66.0	
35392	4220	grace	mitchell	EX709AR	False	False	20.0	
35393	13086	oliver	hall	NW277BU	False	False	58.0	
35394	29494	reece	chafhan	SO433HB	False	False	41.0	

35395 rows × 10 columns

There are records with multiple linked accounts:

In [57]:
```
all_customers[all_customers["other_customer_ids"].notnull()].head()
```

Out[57]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_crm
1	31266	harley	palmer	HR250EJ	False	True	33.0	
5	26434	bailey	richardson	SW988AF	False	True	31.0	
6	28961	skye	johnson	M80NA	False	True	54.0	
10	12586	max	moore	M902XX	False	True	20.0	
12	22825	alicia	wood	SO879UN	False	True	24.0	

What we want now is for each duplicate combination, mark one record as the "main" so that when we count the "main" records we get a sense of how many **unique** values there are.

First we need to remove each record from its own duplicate list

In [58]:
```
def remove_own_record(row):
    ids = list(row["other_customer_ids"])
    ids.remove(row["customer_id"])
    return ids

all_customers.loc[all_customers["other_customer_ids"].notnull(), "duplicate_custome
    all_customers[all_customers["other_customer_ids"].notnull()].apply(remove_own_r
)
```

```
all_customers[all_customers["other_customer_ids"].notnull()].head()
```

Out[58]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_crm
1	31266	harley	palmer	HR250EJ	False	True	33.0	
5	26434	bailey	richardson	SW988AF	False	True	31.0	
6	28961	skye	johnson	M80NA	False	True	54.0	
10	12586	max	moore	M902XX	False	True	20.0	
12	22825	alicia	wood	SO879UN	False	True	24.0	

In [59]:
```
all_customers[all_customers["customer_id"].isin([31266, 5411])]
```

Out[59]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_cr
1	31266	harley	palmer	HR250EJ	False	True	33.0	
7208	5411	harley	palmer	HR250EJ	False	True	33.0	

We'll add a "rank" to each row based on when we encounter a combination of name + postcode.

Anything with rank 1 becomes the "main" record.

(We could/should be more sophisticated and choose main records based on, for example, purchase history, completeness of the record, frequency of activity etc.)

In [60]:
```
# first, add a rank per firstname-surname-postcode combination
all_customers["rank"] = all_customers.groupby(columns_to_consider).cumcount()+1

all_customers[all_customers["customer_id"].isin([31266, 5411])]
```

Out[60]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_cr
1	31266	harley	palmer	HR250EJ	False	True	33.0	
7208	5411	harley	palmer	HR250EJ	False	True	33.0	

In [61]:
```
all_customers.loc[all_customers["rank"] == 1, "is_main"] = True
all_customers["is_main"] = all_customers["is_main"].fillna(False)

all_customers = all_customers.drop(columns="rank")

print(f"Total customers in DB: {len(all_customers)}")
print(f"Of which {len(all_customers[all_customers['is_main']])} are unique/main rec
```

```
Total customers in DB: 35395
Of which 27394 are unique/main records
```

Assumptions/limitations:

- currently the first instance has become the "main" record, this could be improved to choose the one with the highest amount of spend for example (assuming there are linked purchases)
- we assumed all data has to match exactly, so there could be duplicates e.g. with misspellings/typos

Record linkage

In [62]:
```
import recordlinkage

# index the dataframes
indexer = recordlinkage.Index()
indexer.block('postcode')
candidate_links = indexer.index(all_customers.set_index("customer_id"))

# set up the comparison rules
compare = recordlinkage.Compare()
compare.string('first_name', 'first_name', method='damerau_levenshtein', threshold=
compare.string('surname', 'surname', method='damerau_levenshtein', threshold=0.85,
compare.exact('postcode', 'postcode', label="postcode")

# create the comparison vectors
compare_vectors = compare.compute(candidate_links, all_customers.set_index("custome
```

This is now every combination of customer IDs and how many criteria matched

In [63]:
```
compare_vectors
```

Out[63]:

customer_id_1	customer_id_2	first_name	surname	postcode
7523	**7466**	0.0	0.0	1
7492	**7466**	0.0	0.0	1
	7523	0.0	0.0	1
7518	**7466**	0.0	0.0	1
	7523	0.0	0.0	1
...	**...**	...	...	...
17659	**-7621**	1.0	1.0	1
	-7924	0.0	0.0	1
	17654	0.0	0.0	1
	17680	0.0	0.0	1
	27928	0.0	0.0	1

1020864 rows × 3 columns

Extract just the perfect matches, i.e. 3/3 criteria passed

In [64]:
```
matches = compare_vectors[compare_vectors.sum(axis=1) == 3]
matches
```

Out[64]:

customer_id_1	customer_id_2	first_name	surname	postcode
7469	**28769**	1.0	1.0	1
28583	**7468**	1.0	1.0	1
32670	**7485**	1.0	1.0	1
31690	**7476**	1.0	1.0	1
32600	**7516**	1.0	1.0	1
...	**...**	...	...	...
17677	**32903**	1.0	1.0	1
-489	**30716**	1.0	1.0	1
-2407	**17682**	1.0	1.0	1
17680	**-1098**	1.0	1.0	1
17659	**-7621**	1.0	1.0	1

7148 rows × 3 columns

Now merge this back to the customer data so we have all the customer records alongside the duplicate pairs

In [65]:
```
match_df = pd.DataFrame(
    data=matches.index.tolist(),
    columns=["customer_id_1", "customer_id_2"]
)

matched = all_customers.merge(match_df, left_on="customer_id", right_on="customer_i
matched = matched.merge(match_df, left_on="customer_id", right_on="customer_id_2",

matched
```

Out[65]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_
0	7466	eve	richards	HR90PT	False	True	45.0	
1	31266	harley	palmer	HR250EJ	False	True	33.0	
2	534142828	NaN	NaN	NaN	False	True	NaN	
3	1035	luca	gibson	DE256NH	False	True	30.0	
4	6985	mia	rogers	HR662RP	False	True	43.0	
...	...	...	...	...	...	...	...	
35804	14295	erin	morgan	NW481EN	False	False	63.0	
35805	28025	aaron	harris	SO265RP	False	False	66.0	
35806	4220	grace	mitchell	EX709AR	False	False	20.0	
35807	13086	oliver	hall	NW277BU	False	False	58.0	
35808	29494	reece	chafhan	SO433HB	False	False	41.0	

35809 rows × 16 columns

In [66]:
```
matched[matched["customer_id"] == 30730]
```

Out[66]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_cr
4829	30730	harvey	robertson	NW436BL	False	True	34.0	
4830	30730	harvey	robertson	NW436BL	False	True	34.0	
4831	30730	harvey	robertson	NW436BL	False	True	34.0	

Reduce multiple records into a single list (like `duplicate_customer_ids`)

In [67]:
```
def merge_duplicates(group):
    duplicate_list = []
```

```
    if np.isnan(group["customer_id_1_y"].values[0]) == False:
        duplicate_list.extend(group["customer_id_1_y"].tolist())
    if np.isnan(group["customer_id_2_x"].values[0]) == False:
        duplicate_list.extend(group["customer_id_2_x"].tolist())
    if len(duplicate_list) > 0:
        return sorted(list(set([int(x) for x in duplicate_list])))
    return np.nan

linkages = (
    matched
    .groupby("customer_id")
    .apply(merge_duplicates)
    .reset_index(name="linked_duplicates")
)

linkages.head()
```

Out[67]:

	customer_id	linked_duplicates
0	-8300	NaN
1	-8299	[5652]
2	-8298	NaN
3	-8297	NaN
4	-8296	NaN

Merge this back to the main dataset to see the difference

In [68]:
```
all_customers = all_customers.merge(linkages, on="customer_id", how="left")
all_customers.head()
```

Out[68]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_crm_c
0	7466	eve	richards	HR90PT	False	True	45.0	F
1	31266	harley	palmer	HR250EJ	False	True	33.0	
2	534142828	NaN	NaN	NaN	False	True	NaN	F
3	1035	luca	gibson	DE256NH	False	True	30.0	F
4	6985	mia	rogers	HR662RP	False	True	43.0	F

Sort original duplicate customer IDs too so we can compare results

In [69]:
```
all_customers["duplicate_customer_ids"] = all_customers.loc[all_customers["duplicat
```

In [70]:
```
(
    all_customers[(all_customers["duplicate_customer_ids"].notnull())
                & (all_customers["linked_duplicates"].notnull())
                & (all_customers["duplicate_customer_ids"] != all_customers["linked_
)
```

Out[70]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_
2197	10383	scarlett	jackson	M284ZB	False	True	35.0	
7203	19549	summer	anderson	RG546PY	False	True	47.0	
9445	16969	georgia	scott	PO466DY	False	True	9.0	
13277	5390	josh	simpson	HR235FS	False	True	81.0	
14692	32354	josh	simpson	HR235FS	False	True	81.0	
16230	19551	summer	anderson	RG546PY	False	True	38.0	
21022	16966	georgia	scott	PO466DY	False	True	44.0	
24809	10386	scarlett	jackson	M284ZB	False	True	12.0	
28568	-3608	isabella	harrison	NW908RT	True	True	NaN	
31139	-6179	lily	chapman	LS238QF	True	True	NaN	
33507	15377	isabella	harrison	NW908RT	False	False	5.0	
33967	7936	lily	chapman	LS238QF	False	False	28.0	
34662	7934	lily	chapman	LS238QF	False	False	56.0	

In [71]:
```
all_customers[all_customers["customer_id"].isin([10383, 10386, 27536, 28786])]
```

Out[71]:

	customer_id	first_name	surname	postcode	is_guest	in_purchase_data	age	in_c
2197	10383	scarlett	jackson	M284ZB	False	True	35.0	
10596	27536	scariett	jackson	M284ZB	False	True	12.0	
19538	28786	sgarlett	jagkson	M284ZB	False	True	35.0	
24809	10386	scarlett	jackson	M284ZB	False	True	12.0	

Finally, count the difference in the number of duplicates based on the two methods

In [72]:
```
print(len(all_customers[all_customers["is_main"] == False]))
print(len(all_customers[all_customers["linked_duplicates"].notnull()]))
```

```
8001
13689
```